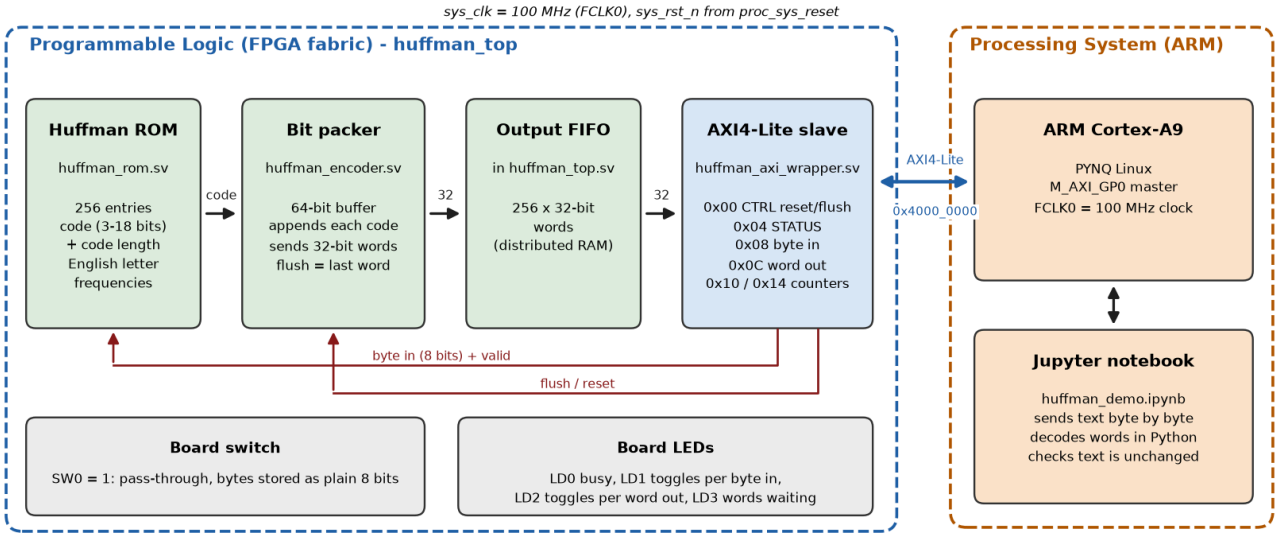
Experiment 2: Huffman Compression Accelerator

FPGA Build Challenge 2026 - Beginner level - AMD PYNQ-Z2 (Zynq-7000 XC7Z020)

1. Objective

Plain text stores every character in 8 bits, although some letters are used far more often than others. Huffman coding gives short codes to common characters and long codes to rare ones, so the same text needs fewer bits and can still be decoded exactly (lossless compression). The objective of this experiment is to build a Huffman encoder in the FPGA that the ARM processor can use over AXI4-Lite: the processor writes text one byte at a time, the hardware looks up each code, packs the variable-length codes into 32-bit words and returns them. The result is checked by decoding it again in software.

2. Block Diagram



Experiment 2 - Huffman Compression Accelerator on the PYNQ-Z2 (Zynq XC7Z020)

Figure 1 - System block diagram

The processor writes a byte to the AXI4-Lite slave. The byte addresses the Huffman ROM, which returns the code (3 to 18 bits) and its length one clock later. The bit packer appends the code to a 64-bit buffer; whenever 32 or more bits are waiting, the lowest 32 bits are sent to a 256-word output FIFO. A flush command sends the last partial word. The processor reads the words and two counters (bytes in, bits out) back through the same slave. Switch SW0 selects a pass-through mode in which each byte is stored as its plain 8 bits.

3. RTL Design

3.1 Module hierarchy

```
system_wrapper               (Vivado block design)
 |-- processing_system7_0    ARM Cortex-A9, gives FCLK0 = 100 MHz and M_AXI_GP0
 |-- rst_ps7_0_100M          reset synchroniser
 |-- ps7_0_axi_periph        AXI interconnect / protocol converter
 +-- huffman_top_0           huffman_top.sv  (output FIFO, counters, LEDs)
      |-- axi_wrapper        huffman_axi_wrapper.sv
      +-- encoder            huffman_encoder.sv
           +-- rom_inst      huffman_rom.sv  (generated by generate_huffman_table.py)
```

3.2 Modules

Module (file)	What it does
generate_huffman_table.py	Python script that builds the code table. It gives each byte a weight from English letter frequencies (every other byte weight 1), runs Huffman's algorithm (repeatedly merge the two lightest nodes) to get a code length per byte, then hands out canonical codes so no code is the start of another. It writes the ROM, the testbench table and the JSON table used by the decoder, so all three always agree.
huffman_rom (huffman_rom.sv)	A 256-entry case statement: byte in, code and code length out, registered (one clock). The codes are stored bit-reversed so that the first code bit ends up in the lowest free bit of the buffer.
huffman_encoder (huffman_encoder.sv)	Delays the byte valid signal by one clock to match the ROM. A multiplexer picks the ROM code or, in pass-through mode (SW0), the raw byte with length 8. The code is shifted left by bit_count and ORed into a 64-bit buffer. If the new count reaches 32 the low 32 bits leave as word_out and the buffer shifts down by 32. A flush sends any remaining bits as a last word and pulses done.
huffman_axi_wrapper (huffman_axi_wrapper.sv)	AXI4-Lite slave with six registers (table below). Writes to CTRL and DATA IN become one-clock pulses. The done pulse is latched in STATUS until the next command so software cannot miss it. Reading DATA OUT pops one word from the FIFO.
huffman_top (huffman_top.sv)	Connects the blocks. Holds the 256 x 32-bit output FIFO (distributed RAM, with a word count), the bytes-in and bits-out counters, and drives the LEDs: LD0 busy, LD1 toggles per byte in, LD2 toggles per word out, LD3 lit while words are waiting.

3.3 Register map (base address 0x4000_0000)

Offset	Name	Access	Meaning
0x00	CTRL	W	bit 0 = reset the engine and counters, bit 1 = flush the last bits
0x04	STATUS	R	bit 0 busy, bit 1 done, bit 2 FIFO empty, bit 3 FIFO full, bits 31:16 words waiting
0x08	DATA IN	W	one byte to compress
0x0C	DATA OUT	R	next compressed word; reading removes it from the FIFO
0x10	BYTES IN	R	number of bytes fed in
0x14	BITS OUT	R	number of compressed bits produced

3.4 Code table

Characters	Code length
space, e	3 bits
t, a, o, i, n, s	4 bits
h, r, d, l	5 bits
c, u, m, w, f, g, y, p, b, new line, full stop, comma	6 bits
v, k	7 bits
digits, x, j, q, z	8 to 11 bits
capital letters, symbols such as ; ( ) { } =	17 to 18 bits

Built from English letter frequencies, the table averages 4.54 bits per character on English text (the script prints this when it runs).

3.5 RTL schematic

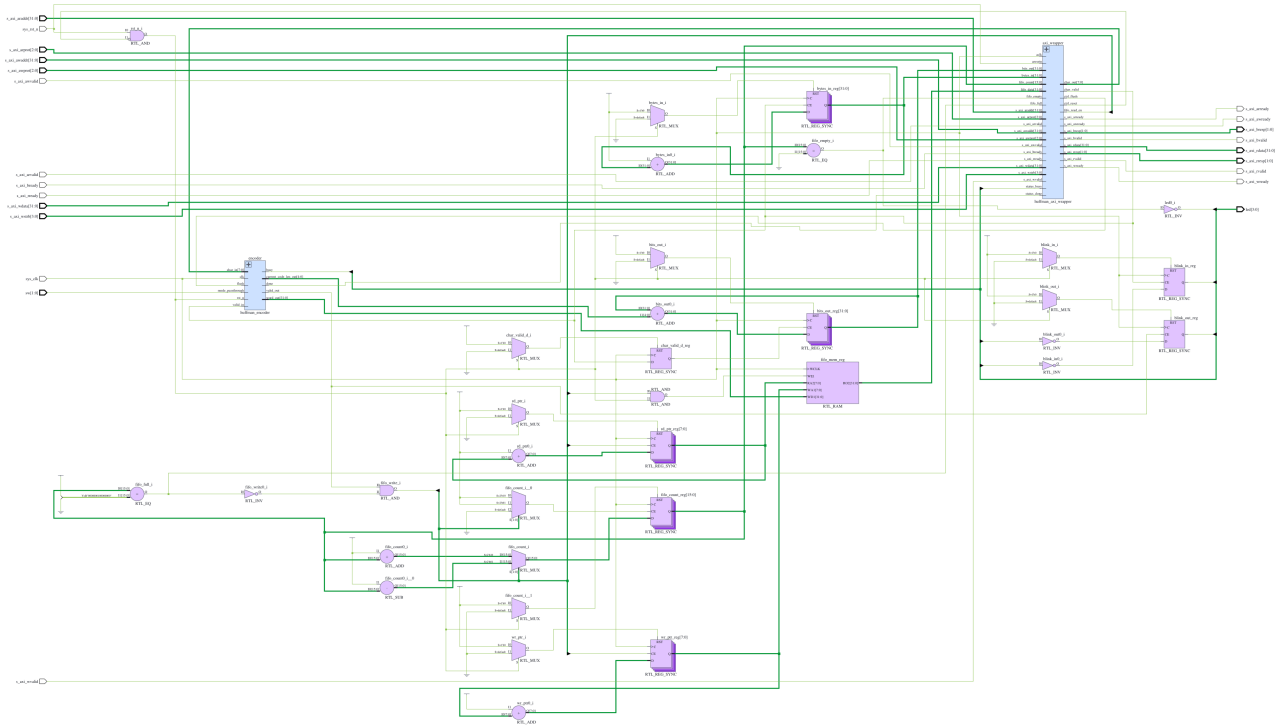


Figure 2 - Elaborated RTL schematic of huffman_top (Vivado)

4. Simulation Results

The design was simulated with Icarus Verilog using the testbench *huffman_tb.sv*. It compresses the 32-byte message "hello world, huffman on a pynq!!", flushes, and checks the byte counter, the bit counter, the number of words and every word against the expected output built from the same code table. All checks passed:

```
bytes in  = 32 (expected 32)
bits out  = 179 (expected 179)
status    = 00060002, 6 words waiting
compressed 32 bytes (179 bits) into 6 words, all correct
PASS: all Huffman accelerator checks passed
```

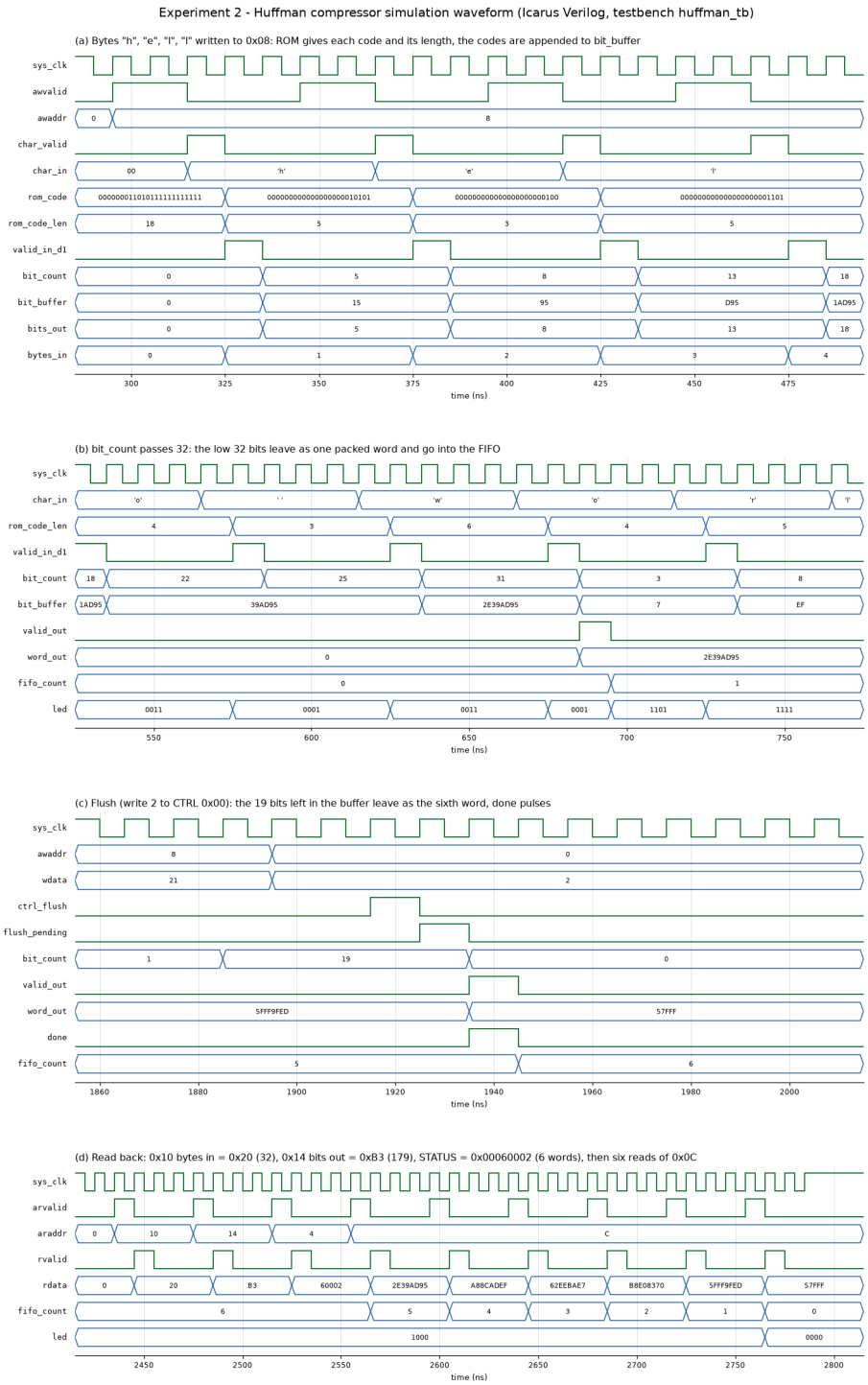


Figure 3 - Simulation waveform (details in Simulation/simulation_report.pdf)

5. Hardware Implementation

5.1 Build

The design was built with Vivado 2026.1 for the xc7z020clg400-1 device using build.tcl. The block design connects the Zynq processing system, a reset block, an AXI interconnect and huffman_top. The constraints file (huffman.xdc) assigns the four LEDs and the two switches.

Resource	Used	Available	Utilisation
Slice LUTs	928	53,200	1.74 %
of which LUT as memory (output FIFO)	236	17,400	1.36 %
Slice registers (flip-flops)	746	106,400	0.70 %
Block RAM tiles	0	140	0 %
Bonded I/O pins	5	125	4.00 %
Global clock buffers	1	32	3.13 %

Timing (100 MHz clock)	Value	Status
Worst setup slack (WNS)	+2.934 ns	**met**
Worst hold slack (WHS)	+0.087 ns	**met**
Failing endpoints	0 of 3185	**met**

5.2 Test on the board

The bitstream was loaded into the PYNQ-Z2 with SW0 down (compression mode). Three texts were sent through the compressor exactly as the notebook huffman_demo.ipynb does it: reset, write every byte to 0x08, read words from 0x0C as they appear, flush and read the counters. For each text the returned words were compared bit for bit with a Python model of the encoder, decoded back to text, and the size was compared with the software compressor zlib at its best setting.

Text	Bytes in	Bits out	Bits / char	FPGA size	zlib size	Result
Notebook text (lower-case English)	600	2764 in 87 words	4.61	57.6 %	21.3 %	**PASS**
English paragraph (normal case)	897	4066 in 128 words	4.53	56.7 %	56.9 %	**PASS**
Verilog source code	584	3449 in 108 words	5.91	73.8 %	42.1 %	**PASS**

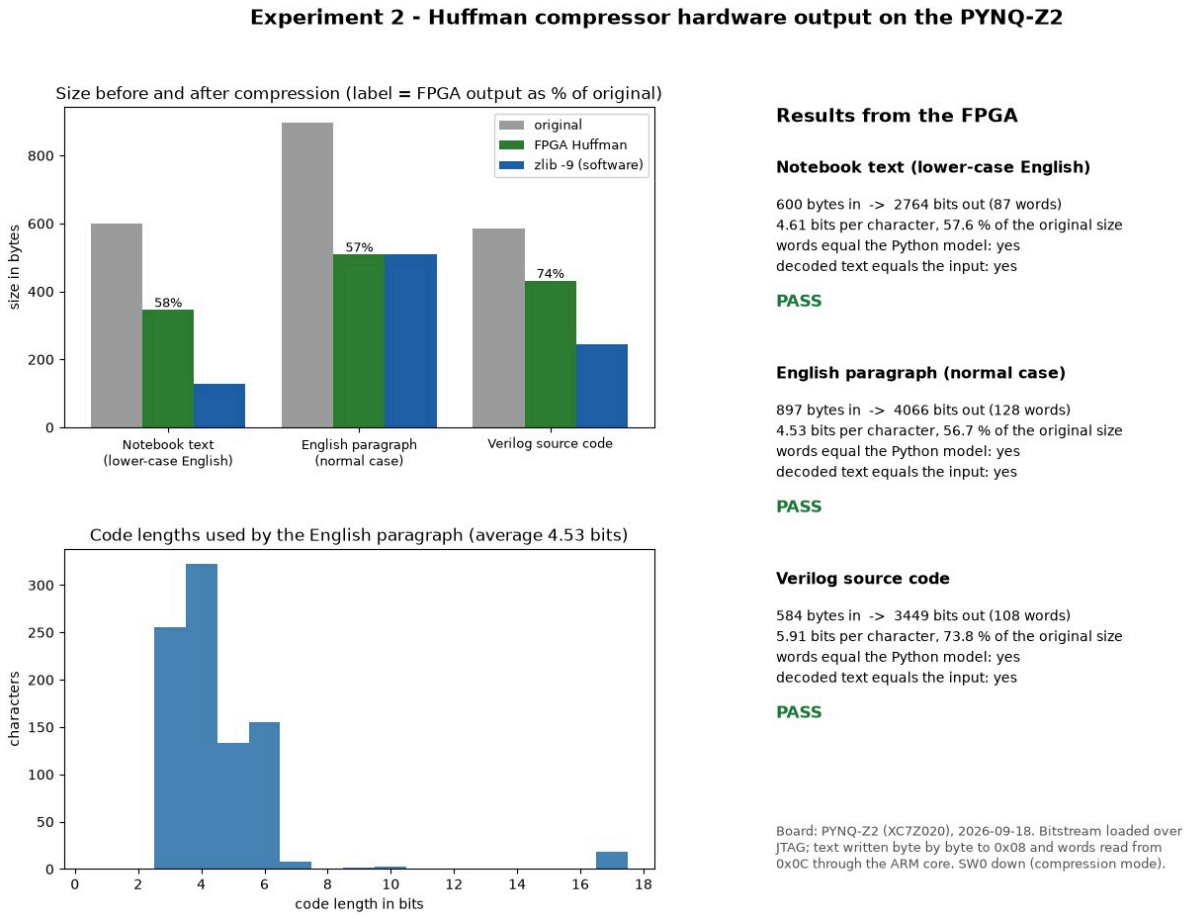


Figure 4 - Compression results measured on the FPGA

Run of the Jupyter notebook on the board

The steps of the notebook huffman_demo.ipynb (Jupyter folder) were also run on the PYNQ-Z2 with PYNQ, using the same bitstream. tools/execute_all_on_pynq.py runs them on the board over SSH and copies the log and the graph back, and tools/embed_nb_outputs.py puts them into the notebook. It sent the same 600-byte text and got back 87 words holding 2764 bits (57.6 % of the original size, 1.74 to 1), exactly the same as the run above, and the decoded text matched the input byte for byte. Through PYNQ the transfer took 0.04 s instead of about 2 s over JTAG. The log and the graph the notebook drew are in Jupyter/output.

```
[*] Flashing FPGA bitstream for Experiment 2 (Huffman)...
[+] FPGA Flashed! DONE LED active.
Overlay loaded, compressor at 0x40000000
Sent 600 bytes (4800 bits) in 0.04 s
Got back 87 words holding 2764 bits
Compression ratio: 1.74 to 1 (57.6 % of original size)
Decoded text starts: the quick brown fox jumps over the lazy dog. huffman coding
PASS: all 600 bytes came back exactly as they went in
```

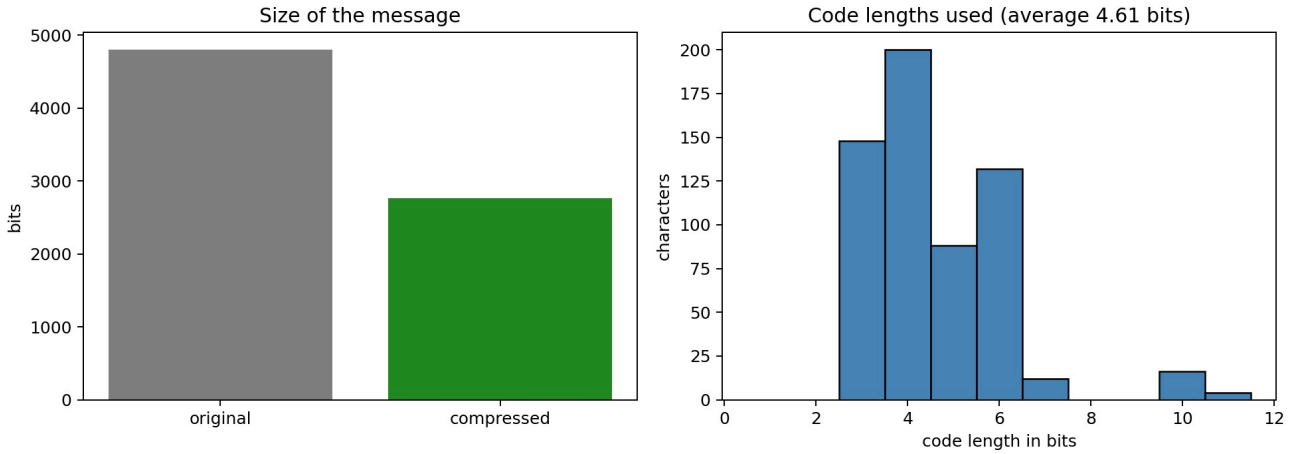


Figure 5 - Graph drawn by huffman_demo.ipynb on the board

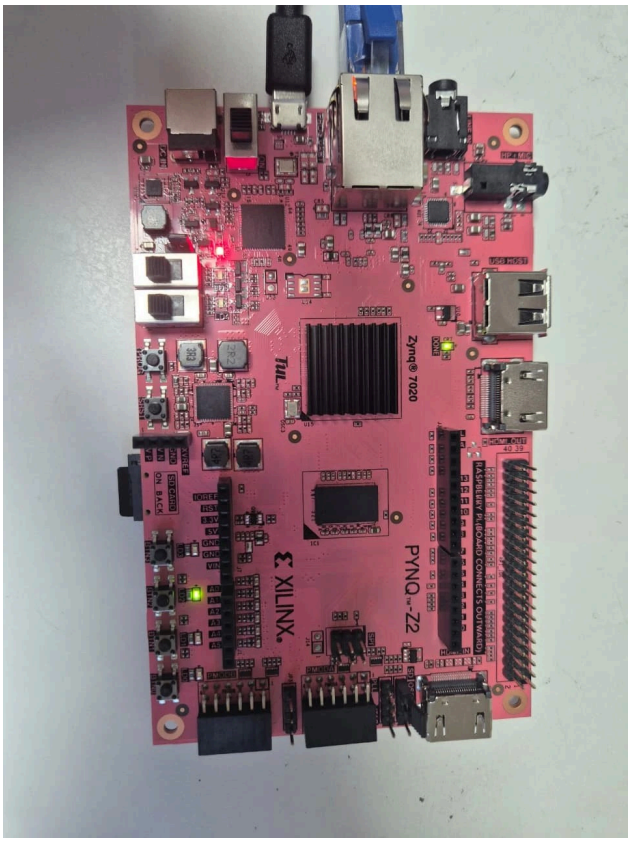


Figure 6 - PYNQ-Z2 board setup: micro-USB (power and JTAG/UART), Ethernet to the PC, PYNQ Linux on the SD card

For all three texts the hardware output was identical to the model and decoded back to exactly the original text. English text comes out at about 4.5 to 4.6 bits per character, 57 % of its original size, which matches the average code length of the table. On the English paragraph the simple FPGA encoder is as good as zlib. zlib is much better on the notebook text because that text repeats the same sentence four times, and zlib also replaces repeated phrases (LZ77), which a fixed Huffman table cannot do. Source code compresses less (74 %) because symbols such as ; ( ) = get 17- or 18-bit codes in a table made for English; this shows that the table should match the kind of data being compressed.

6. Applications

- **IoT and sensor nodes:** compress text logs or messages before sending them over a slow or battery-powered radio link.
- **Data storage and logging:** store more records in the same flash memory.

- **Building block of standard formats:** Huffman coding is the final stage of DEFLATE (ZIP, PNG, gzip) and of JPEG, so the same encoder can be part of a larger hardware compressor.
- **Offloading the processor:** the CPU only moves bytes while the FPGA does the bit-level packing.

7. Conclusion

A Huffman compression accelerator was designed, simulated and implemented on the PYNQ-Z2. It uses under 2 % of the FPGA's LUTs, meets timing at 100 MHz with 2.9 ns to spare, and on the board it compressed English text to about 57 % of its size with the output decoding back perfectly.

Learning outcomes: how Huffman's algorithm builds a prefix-free code; how to generate a hardware lookup table from a script so that hardware, testbench and software always agree; packing variable-length codes into fixed 32-bit words with a shift buffer; writing a FIFO and an AXI4-Lite slave; and checking hardware against a software model.

Future scope: load the code table over AXI so it can be built for each file (adaptive Huffman); add an LZ77 stage in front to remove repeated phrases like zlib does; add a hardware decoder; and use AXI-Stream with DMA so the processor does not have to write every byte itself.